

cate the request

- More on ArgoController: receives events from Argo. When the source language files have completed translation and are ready in Argo, Argo will make a call to the GitLab Integration informing what has completed, which it receives via the ArgoController. From there, the GitLab Integration will grab the translated file, create a branch and MR if necessary, and the commit to that branch.

2. (GitLab Event) GitLabWebhookService will take the event, convert that data, and check if it's relevant. If the checks find it is relevant, then it's added to the queue

- Purpose: Filter out most of the junk events, convert the data into a more manageable form, handle events immediately, prevent the queue and database being filled with useless events

3. (Argo Event) ArgoStepService will take the event and just add it to the queue

- Purpose: All Argo events are relevant so this has no use, but if in the future we need to do preprocessing like with the GitLab events, then this serves as a convenient place to do so

An in-depth diagram of Queue Service

ArgoController

ArgoController receives events from Argo. When the source language files have completed translation and are ready in Argo, Argo will make a call to the GitLab Integration informing what has completed, which it receives via the ArgoController. From there, the GitLab Integration will grab the translated file, create a branch and MR if necessary, and the commit to that branch. Future developments could have Argo making various other calls, such as querying the if the Translation MR has been merged to display in Argo. Such possible future calls would be added to the ArgoController.

Event Queuing

The QueueServiceImpl implements a sophisticated event queuing system that handles concurrent processing while maintaining project-specific FIFO (First-In-First-Out):
In computing and in systems, the queue is processed first.
ordering. The system processes events after their initial pre-processing within HTTP requests, managing both database persistence and in-memory queuing.

Event Reception and Storage

When an event is received by the QueueService:

- The event data is stored into the database for logging and retry capabilities,
- An `EventRunnable` object is created, containing both the event data and its execution function, and the function will be executed on a new thread and use the provided event data,
- The `EventRunnable` is submitted to a `GroupedExecutingThreadPool`.

Queuing Implementation

The system uses a two-level threading approach:

- Primary ThreadPoolExecutors (dequeuer): Receives all incoming EventRunnables in their initial

- Secondary ThreadPoolExecutors (executors): An array of executors where each entry is dedicated to processing events for a specific GitLab Project

- Events within the same Project must be processed synchronously in FIFO (First-In-First-Out) order, (~:~text=In%20computing%20and%20in%20systems,the%20queue%2C%20is%20processed%20first.)
- Different Projects can be processed concurrently, up to a maximum of 5 Projects at once,
- If an event is next in queue for a Project that cannot be processed (due to the 5-Project limit), it will block all subsequent events regardless of their Project,
- Events that arrived before the blocked event continue processing normally.

The system maintains event status in the database with the following states:

- ## Execution Process

- ## Purpose and Benefits

- Merge Request closed

1. Get Merge Request data from GitLab API
 - Purpose: Not all needed data is provided in the Event. It will also get the latest data in case there were changes since the event was called.
2. Get a list of files needing translation
 - This depends on the translation MR and the contents of the changed files in the MR.

3. Get the contents of those files
4. Create the Argo Request for those files
5. Add cards (see definition below) and allocate assets in the Request
 - Purpose: Provide additional data like links, and organize the Schedule so it's ready to launch, or can be auto launched.
 - If any exception happens, delete the Argo Request so it's not incomplete.

Quick notes:

What data is contained in an Argo Request?

You can see an example here.

Requests contain data about GitLab, the TMS it's connected to, and who worked on the Request such as comments and assignees. The following information from the GitLab integration is put into a Request:

- Project ID and Path,
- Merge Request ID and title and URLs,
- Components,
- File names and contents of files that will undergo translation.

What is a "card?"

In a Request, you can have fields with one single value. Such as "Project ID". Cards are like a mini table rows that just hold data. Each card can have fields with single values, but multiple cards can be added to a Request. Their values are tabulated onto a table within the Request, where each row is a card, and each column is one field. We use Cards for the GitLab Integration to list all relevant URLs so it makes it easier for Request viewers to go from the Request screen to the MRs.

Argo Commit to GitLab

GitLabMergeRequestServicehandleCommitToTranslationMREvent Whenever translation has completed on a target file, Argo attempts to commit the file back to GitLab. It will do this one file at a time.

1. Get details from Argo about the Request and the Plan and its content,
2. Get details from GitLab about the Project,
3. Get the Translation YAML and decide where to place the target files based on this
 - Will attempt to find the file path in the Argo Translation YAML (the configuration file that has the information on folder locations) for the source file, then remap to the target language.
4. Commit the file and its contents to a Translation MR
5. Update Argo that the step is completed

Deployment Phase(s)

The deployment for the Argo\<->GitLab Integration will be handled in two phases.

Phase 1 - Spartan

The first phase will have the application deployed in Spartan's environment to expedite the process of running and testing the application. It is only a temporary step until the application can ultimately be deployed to GitLab's environment.

The application is built using GitLab's CI/CD Pipeline. The artifact is then downloaded and deployed by the Spartan Software engineers.

The application is deployed in a Rocky Linux 8 OS running on AWS EC2. It's runs on Java 17, with an Nginx proxy. It runs on the same instance as the Argo application. The nginx application is configured to allow only the endpoint for GitLab webhook events to access the application. All other endpoints can only be accessed within the box via localhost. The database is hosted in Amazon RDS, with the only allowed connections being that specific EC2 instance and engineers on the VPN. There is both a production version of this setup, and a test version of this setup. So there are two of each of the databases and EC2 instances, but prod and test cannot communicate with one another.

Phase 2 - GitLab

- Pipelines:

1. When the GitLab environment is ready, it will work the same for building in the Pipelines from the same repository.
2. However, pipelines would run to build the executable java(.jar) plus build the docker container image, that includes Java 17 and nginx to act as a proxy.

- Google Cloud:

1. To configure the database part of the Argo-GitLab integration service, we can install and deploy the database services on Google Cloud Sandbox, and use Cloud SQL to connect from runway.
2. More implementation details are documented [here](#) and [here](#).

- Runway:

1. It get's deployed in Runway from the docker image following Runway onboarding (onboarding details are documented [here](#) and [here](#)).
2. Runway service connects to the cloudSQL instance outside of the runway GCP projects using private IP + private service connect

Alternative Solutions

One quick solution is to build a linux server in to Google Cloud and install the java application and the rest of the database configuration on cloud. We can call this as Traditional VM based deployment whether it's done on the cloud or somewhere inside GitLab:

- Quick initial setup using Google Cloud VM with Java application and database
- Minimal initial configuration required
- Lower initial development overhead

However, it has some serious limitations:

- Deployment challenges:
 - Manual deployment process requiring executable JAR rebuilds
 - Lack of zero-downtime deployment capabilities
 - No automated rollback mechanisms
 - Risk of configuration drift between environments
- Operational concerns:
 - Pipeline interruption during deployments
 - Potential data loss during Argo communications
 - Limited scalability options
 - Higher maintenance overhead
- Monitoring and recovery:
 - Limited built-in monitoring capabilities
 - Complex disaster recovery procedures
 - No automatic failover options

SECTION: engineering/architecture/design-documents/gitlab_xray_rag/_index.md

{{< engineering/design-document-header >}}

Group ~"group::global search" is leading effort to build RAG at GitLab. Because this is a global effort, it is in the spirit of efficiency and collaboration values for ~"group::code creation" to join that effort and integrate Repository X-Ray data into GitLab RAG. Doing so should not only result in more efficient resource allocation, but also enable other AI features to integrate and reuse Repository X-Ray data, for example users could ask questions to GitLab Duo Chat that could be answered with X-Ray report data.

Goal

Integrate the existing Repository X-Ray scan flow with the RAG platform.

Proof of Concept

Proof of concept has been built at merge request 144715. This MR contains a large amount of information that can be helpful during implementation.

Implementation

Current state

Currently, Repository X-Ray does not use semantic search. The MVC approach naively selects first 50 entities from the X-Ray report and includes them into a code generation request.

For more details, see the following diagrams.

<details><summary> <bold>Current state of Repository X-Ray diagrams</bold> </summary>

The Repository X-Ray scan is processed as shown on the following diagram:

```

mermaid
sequenceDiagram
    actor USR as User
    participant RN as GitLab Runner

    participant GLR as GitLab Rails
    participant PG as GitLab PostgreSQL DB
    participant AIGW as AI Gateway

```

```

USR->>GLR: commits changes <br> to a package manager file <br>eg. Gemfile.lock
GLR->>+RN: triggers Repository X Ray CI scanner job
loop for each batch of packages
    RN->>GLR: Request packages description by AI
    GLR->>AIGW: Forward request for packages description
    AIGW->>GLR: Packages description
    GLR->>RN: Forwards packages description
end
RN->>-GLR: Repository X Ray report
GLR->>+GLR: triggers Repository X Ray ingestion background job
GLR->>-PG: upserts xrayreports record

```

The report is later used as shown in the following diagram:

```

mermaid
sequenceDiagram
    actor USR as User
    participant IDE
    participant PG as GitLab PostgreSQL DB
    participant GLR as GitLab Rails
    participant AIGW as AI Gateway

    USR->>+IDE: types: "35; generate a function that transposes a matrix"
    IDE->>+GLR: trigger code generation for line "35; generate function
    GLR->>PG: fetch X Ray report for project and language
    PG->>GLR: xrayreports record
    GLR->>GLR: include first 50 entities from xray report into code generation prompt
    GLR->>-AIGW: trigger code generation "35; generate function

```

</details>

Desired outcome

After this effort is completed, the Repository X-Ray scan is processed in the following manner:

```

mermaid

```

sequenceDiagram

actor USR as User
participant RN as GitLab Runner
participant GLR as GitLab Rails
participant ES as Elasticsearch
participant PG as GitLab PostgreSQL DB
participant AIGW as AI Gateway

USR->>GLR: commits changes
 to a package manager file
eg. Gemfile.lock

GLR->>+RN: triggers Repository X Ray CI scanner job

loop for each batch of packages

 RN->>GLR: Request packages description by AI

 GLR->>AIGW: Forward request for packages description

 AIGW-->>GLR: Packages description

 GLR-->>RN: Forwards packages description

end

RN->>-GLR: Repository X Ray report

GLR->>+GLR: triggers Repository X Ray ingestion background job

GLR->>PG: Fetch currently stored packages for given repository and language

PG-->>GLR: List of repository packages

GLR->>GLR: Group packages into lists58; "skip", "add", "remove"

GLR->>PG: Insert new packages from "add" list for given repository

GLR->>PG: Delete packages from "remove" list for given repository

rect rgb(0, 223, 0, .1)

note right of RN: Embeddings flow

 opt with Elasticsearch available on an instance

 GLR->>ES: Check if packages from "add" list already exists in ES index

 ES-->>GLR: List of existing packages from "add" list

 GLR->>GLR: Remove already indexed packages from "add" list

 loop for each remaining package in "add" list

 GLR->>AIGW: Request embeddings for package description

 AIGW-->>GLR: Embeddings for package description

 end

 GLR->>ES: Create documents for packages in "add" list in ES index

GLR->>PG: Check if packages form "remove" list exists in any repository

PG-->>GLR: List of packages from "remove" group still in use

GLR->>GLR: Filter packages that are still in use from "remove" list

GLR->>ES: Delete all remaining packages in "remove" list from ES index

end

end

Later on, the Repository X-Ray report is used as follows:

mermaid

sequenceDiagram

actor USR as User
participant IDE

participant GLR as GitLabRails
participant ES as Elasticsearch
participant PG as PostgreSQL
participant AIGW as AI Gateway

```
USR->>+IDE: types: "35; generate method that fetches <br>top charts from Spotify"
IDE->>+GLR: trigger code generation for "35; generate method <br>that fetches top charts
from Spotify"
  alt with Elasticsearch available on an instance
    rect rgb(0, 223, 0, .1)
    note left of GLR: new embeddings flow
      GLR->>+AIGW: fetch embedding for instruction "in utils.js generate method that ..."
      AIGW-->>-GLR: embeddings vector for instruction
      GLR->>PG: fetch packages for given repository and programming language
      PG-->>GLR: List of repository package names for given language
      GLR->>ES: fetches packages documents using KNN for instruction embeddings vector
<br> from filtered list using repository package names
      ES-->>GLR: spotify/web-api-ts-sdk - A package that wraps ...
      GLR->>AIGW: code generation request with prompt including <br>spotify/web-api-ts-sdk
- A package that wraps...
    end
  else
    rect rgb(128, 128, 128, .1)
    note left of GLR: current flow as fallback
      GLR->>PG: fetch X Ray report for repository and language
      PG-->>GLR: xrayreportpackages records
      GLR->>AIGW: code generation request with prompt first 50 <br> entities from xray
report
    end
  end
end
```

Required changes

On X Ray write path

1. Create new AI Gateway endpoint that generate embeddings

Embeddings for GitLab documentation are requested directly from GitLab Rails. That approach limits embeddings availability to GitLab.com only, and this is also misaligned with the architecture blueprint that outlines AI Gateway as a provider of AI services for GitLab features (for more information, see the related AI Gateway epic 13024). To avoid similar problems for Repository X-Ray, a new endpoint should be added to AI Gateway API that accepts batches of strings, and responds with embeddings vectors.

In the following iterations, it is expected that the AI Gateway endpoint (as the central point which has complete overview of all connected instances traffic) will enforce rate limiting and manage token consumption. However, from the very start clients will be responsible for correctly handling rate limiting and token exhaustion type errors.

Open question:

Embeddings are generated with the textembedding-gecko model (768 dimensions). When adding a new API endpoint, we might be able to change model if needed, if so we should decide which one. Because Repository X Ray report data has small volume and size (at this moment there are 379 reports on GitLab.com), the decision to switch model and rebuild embeddings data is low cost and can be deferred in order to unblock iteration.

2. Store embeddings vectors for Repository X-Ray report

The current Elasticsearch framework uses ActiveRecord to keep the index up to date. It uses callbacks on create/update/delete to action the corresponding record in Elasticsearch. Because xrayreports table in PostgreSQL main database stores whole reports as JSON blobs in order to persist generated embeddings vectors for each of report item (representing library used in given repository) we would need to either:

- Modify the current Elastic framework as defined in issue 442197.
- If, due to higher urgency, migrate xrayreports table to a new structure, where each record represents a single entity (package/library) in the Repository X-Ray report, which would be compatible with the current Elasticsearch upload pipeline.

The index in Elasticsearch that stores Repository X-Ray packages has the following format:

```
json
index: xray-packages
document: {
  id:
  name:
  language:
  description:
  embedding:
}
```

This index would store:

- Description.
- Name.
- The embeddings generated for the concatenated package name and description.
- Programming language.

This index will be shared between all repositories in a given GitLab instance. However, upon search the list of relevant packages will be filtered using prefetched lists of names of packages that belong to the given repository in a given programming language. That approach should:

- Reduce required storage capacity.
- Improve search performance.
- Reduce AI consumption used to generate embeddings.

Open question

We need to check if descriptions vary significantly depending on version, and decide whether to store records per package and its version, or just one per package.

3. Modify storage on GitLab Rails layer

Right now Repository X-Ray packages are stored in xrayreports table at the main PostgreSQL DB.

```
sql
CREATE TABLE xrayreports (
  id bigint NOT NULL,
  projectid bigint NOT NULL,
  createdat timestamp with time zone NOT NULL,
  updatedat timestamp with time zone NOT NULL,
  lang text NOT NULL,
  payload jsonb NOT NULL,
  filechecksum bytea NOT NULL,
);
```

In order to support sharing embeddings between repositories whilst providing way to remove stale data from Elasticsearch, we must create a new table, xrayreportpackages.

```
sql
CREATE TABLE xrayreportpackages (
  id bigint NOT NULL,
  projectid bigint NOT NULL,
  createdat timestamp with time zone NOT NULL,
  updatedat timestamp with time zone NOT NULL,
  lang text NOT NULL,
  name text NOT NULL,
  version text NOT NULL,
  description text, --nullable filed as with Elasticsearch available this file will not be in
use
);
```

After the new table is created, all reports from xrayreports should be migrated there, and xrayreports should be removed.

4. Modify Repository X-Ray import pipeline

The Repository X-Ray report, after being generated during the CI job, is imported using a background job.

That job uses `AI::StoreRepositoryXrayService` to parse and save the report file into the xrayreports table at the main PostgreSQL DB.

In order to support semantic search for Repository X-Ray, we must apply the following modifications:

1. Load the new Repository X-Ray report from scanner into the list of currentreportpackages.
1. Load the list of packages reported in previous report for the given project stored in xrayreportpackages into the list of previousreportpackages
(For example, SELECT FROM xrayreportpackages WHERE lang = 'ruby' AND projectid = 123).
1. Filter packages from the new X-Ray report into three groups:
 1. skip - a collection of unmodified packages that do not require any action.
 1. add - a collection of new packages that should be added to xrayreportpackages.
 1. remove - a collection of packages that were present in the old report but missing in the new one (previousreportpackages - currentreportpackages).
1. Insert the new records into the PostgreSQL xrayreportpackages for the add list of packages.
1. From the add list, filter out packages that already exist in Elasticsearch. Those are packages that are being used by other repositories, and in this case embeddings and descriptions will be shared.
1. Update and insert all packages that are left in the add list of packages in Elasticsearch.
1. Delete the remove list of packages from PostgreSQL xrayreportpackages for a given project.
1. Create an orphaned list of packages from the remove list by selecting ones of the same name and lang that are not being used by any other project
(For example, SELECT 1 FROM xrayreportpackages WHERE name = 'kaminari' AND lang = 'ruby' LIMIT 1).
1. Remove the orphaned packages from Elasticsearch.

On X-Ray read path

1. Retrieve code generation instruction either from:
 1. IDE:
 1. IDE / LS upon detecting generation should send a code generation instruction (for example, a content of a comment that triggered code generation).
 1. GitLab Rails Code Suggestions API needs to add optional string parameter instruction.
 1. GitLab Rails if due to different priorities the IDE will not be able to deliver instruction on time to unblock this effort, for the sake of the initial iteration, the code generation instruction can be also retrieved by GitLab Rails.
 1. GitLab Rails needs to detect if Elasticsearch is available and:
 1. When Elasticsearch is available:
 1. Fetch list of names of packages that are stored in xrayreportpackages PostgreSQL database for the given project and language.
 1. Use k-nearest neighbor (kNN) search of Elasticsearch filtered with names list of projects packages to retrieve the most relevant context.
 1. When Elasticsearch is not available:
 1. Select the first 50 records from xrayreportpackages in PostgreSQL database aligned with the current state diagram.

SECTION: engineering/architecture/design-documents/git_data_offloading/_index.md

{{< engineering/design-document-header >}}

Summary

Managing Git data storage costs is a critical part of our business. Offloading

Git data to cheaper storage can save on storage cost.

Motivation

At GitLab, we keep most Git data stored on SSDs to keep data access fast. This makes sense for data that we frequently need to access. However, given that storage costs scale with data growth, we can be a lot smarter about what kinds of data we keep on SSDs and what kinds of data we can afford to offload to cheaper storage.

For example, large files (or in Git nomenclature, "blobs") are not frequently modified since they are usually non-text files (images, videos, binaries, etc). Often, Git LFS is used for repositories that contain these large blobs in order to avoid having to push a large file onto the Git server. However, this relies on client side setup.

Or, if a project is "stale" and hasn't been accessed in a long time, there is no need to keep paying for fast storage for that project.

Instead, we can choose to put all blobs of that stale project onto cheaper storage. This way, the application still has access to the commit history and trees so the project browsing experience is not affected, but all files are on slower storage since they are rarely accessed.

If we had a way to separate Git data into different categories, we could then offload certain data to a secondary location that is cheaper. For example, we could separate large files that may not be accessed as frequently from the rest of the Git data and save it to an HDD rather than an SSD mount.

Requirements

There are a set of requirements and invariants that must be given for any particular solution.

Saves storage cost

Ultimately, this solution needs to save on storage cost. Separating out certain Git data for cheaper storage can go towards this savings.

We need to evaluate the solution's added cost against the projected savings from offloading data to cheaper storage. Here are some criteria to consider:

- How much money would we save if all large blobs larger than X were put on HDD?
- How much money would we save if all stale projects had their blobs on HDD?
- What's the operational overhead for running the offloading mechanism in terms of additional CPU/Memory cost?
- What's the network overhead? e.g. is there an extra roundtrip to a different node via the network to retrieve large blobs.
- Access cost, e.g. when blobs would be stored in an object store.

Opaque to downstream consumers of Gitaly

This feature is purely storage optimization and, except for potential performance slowdown, shouldn't affect downstream consumers of Gitaly. For example, the GitLab application should not have to change any of its logic in order to support this feature.

This feature should be completely invisible to any callers of Gitaly. Rails or any consumer should not need to know about this or manage it in any way.

Operationally Simple

When working with Git data, we want to keep things as simple as possible to minimize risk of repository corruption. Keep things operationally simple and keep moving pieces outside of Git itself to a minimum. Any logic that modifies repository data should be upstreamed in Git itself.

Proposal

We will maintain a separate object database for each repository connected through the Git alternates mechanism. We can choose to filter out certain Git objects for this secondary object database (ODB).

Place Git data into this secondary ODB based on a filter. We have options based on filters in Git.

We can choose to place large blobs based on some limit into a secondary ODB, or we can choose to place all blobs onto the secondary ODB.

Design and implementation details

Git

We need to add a feature to `git-repack(1)` that will allow us to segment different kinds of blobs into different object databases. We're tracking this effort in this issue.

Gitaly

During Gitaly housekeeping, we can do the following:

1. Use `git-repack(1)` to write packfiles into both the main repository's object database, and a secondary object database. Each repository has its own secondary object database for offloading blobs based on some criteria.
1. Ensure the `.git/objects/info/alternates` file points to the secondary object database from step (1).

Criteria

Whether objects are offloaded to another object database can be determined based on one or many of the following criteria.

By Tier

Free projects might have many blobs offloaded to cheaper storage, while Ultimate projects have all their objects placed on the fastest storage.

By history

If a blob was added a long time ago and is not referred by any recent commit it can get offloaded, while new blobs remain on the main ODB.

By size

Large blobs are a quick win to reduce the expensive storage size, so they might get prioritized to move to cheaper storage.

Frequency of Access

Frequently used project might remain fully on fast storage, while inactive projects might have their blob offloaded.

Open Questions

How do we delete objects?

When we want to delete an unreachable object, the repack would need to be aware of both ODBs and be able to evict unreachable objects regardless of whether the objects are in the main ODB or in the secondary ODB. This picture is complicated if the main ODB also has an object pool ODB, since we wouldn't ever want to delete an object from the pool ODB.

Potential Solution: Modify Git to delete an object from alternates

We would need to modify repack to give it the ability to delete unreachable objects in alternate ODBs. We could add repack configs `repack.alternates` that tell it how to behave with alternate directories. For example, we could have `repack.alternates.explodeUnreachable`, which indicates to repack that it should behave like -A in any alternate ODB it is linked to.

How does this work with object pools?

When we use alternates, how does this interact with object pools? Does the object pool also offload data to secondary storage? Does the object pool member? In the most complex case this means that a single repository has four different object databases, which may increase complexity.

Possibly we can mark some packfiles as "keep", using the `--keep-pack`

and
--honor-pack-keep
options.

Potential Solution: Do not allow object pools to offload their blobs

For the sake of not adding too much complexity, we could decide that object pools will not offload their blobs. Instead, we can design housekeeping to offload blobs from the repository before deduplicating with the object pool. Theoretically, this means that offloaded blobs will not end up in the object pool.

How will this work with Raft + WAL?

How will this mechanism interact with Raft and the write-ahead log?

The WAL uses hard-links and copy-free moves, to avoid slow copy operations. But that does not work across different file systems. At some point repacks and such will likely also go through the log. Transferring data between file systems can lead to delays in transaction processing.

Ideally we keep the use of an alternate internal to the node and not have to leak this complexity to the rest of the cluster. This is a challenge, given we have to consider available space when making placement decisions. It's possible to keep this internal by only showing the lower capacity of the two storages, but that could also lead to inefficient storage use.

Problems with the design

Added complexity

The fact that we are adding another object pool to the mix adds complexity to the system, and especially with repository replication since we are adding yet another place to replicate data to.

Possible change in cost over time

The cost of the different storage types might change over time. To anticipate for this, it should be easy to adapt to such changes.

More points of failure

Having some blobs on a separate storage device adds one more failure scenario where the device hosting the large blobs may fail.

Alternative Solutions

Placing entire projects onto cheaper storage

Instead of placing Git data onto cheaper storage, the Rails application could

choose to move a project in its entirety to a mounted HDD drive.

Possible optimization

Giving these machines with cheaper storage extra RAM might help to deal with the slow read/write speeds due to the use of page cache. It's not sure though this will turn out to be cheaper overall.

SECTION: engineering/architecture/design-documents/glql/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

{{< engineering/design-document-header >}}

Summary

GitLab Query Language (GLQL) is born out of the need for a unified, powerful, and user-friendly way to query and present data across the entire GitLab platform. As GitLab continues to evolve and expand its feature set, the ability to efficiently access, filter, and visualize information becomes increasingly crucial for users to manage their workflows effectively.

Motivation

GLQL addresses the growing need for a unified and powerful way to access and visualize data across the GitLab platform. As GitLab's feature set expands, GLQL aims to simplify complex data retrieval tasks, enhance user productivity, and provide a consistent querying experience. This initiative positions GitLab as a comprehensive data management solution within the DevOps ecosystem, catering to the evolving needs of developers and organizations managing large-scale projects.

Goals

The primary motivations behind GLQL are:

1. Unified Query Interface: GLQL aims to provide a single, consistent syntax for querying various GitLab objects, starting with issues and expanding to work items.
1. Enhanced Data Accessibility: By embedding GLQL blocks in text editors anywhere across GitLab, GLQL empowers users to access precisely the information they need, when they need it. This is especially essential for large-scale projects that manage complex workflows.
1. Improved Visualization: GLQL's data agnostic presentation layer allows users to customize how query results are displayed, enabling more effective data analysis.
1. AI Integration: With its structured syntax, GLQL is positioned to integrate

seamlessly with GitLab's AI offerings, paving the way for natural language query capabilities and enhancing user productivity.

1. Extensibility: While initially focused on issues and work items, GLQL is designed with the future in mind, allowing for expansion to cover more GitLab objects and use cases as the platform grows.

Proposal

GLQL introduces a versatile model designed to adapt a single query syntax for fetching various objects and presenting them uniformly. A GLQL query consists of two components: the query expression and metadata. The model consists of the following components:

1. Façade: Acts as the entry point for processing GLQL queries, coordinating the flow between various components of the system.
1. Query Compiler: Responsible for parsing and transforming the GLQL query into executable code. It consists of three sub-components:
 - Parser: Breaks down the raw query string into an AST (abstract syntax tree).
 - Analyzer: Validates the parsed query, checking for semantic correctness.
 - Code Generator: Converts the analyzed query into executable code for the Executor.
1. Metadata Parser: Processes the metadata associated with the query, which includes transformation (or aggregation) and presentation instructions. It has two sub-components:
 - JS YAML: Parses YAML-formatted metadata.
 - Fields Parser: Interprets the fields specified in the metadata and transformations to apply, if any.
1. Executor: Runs the compiled query against the API or datastore to retrieve the requested data. Currently, GraphQL is used as a datastore.
1. Transformer: Processes the response data returned by the Executor, applying any transformations specified in the metadata.
1. Presenter: Presents the final transformed data using data-agnostic Vue components.

Here's an example of a GLQL query:

```
md
glql
```

```
project = "gitlab-org/gitlab" AND assignee = currentUser() AND opened = true
```

In the above example, the front matter block enclosed in --- contains metadata. display is an example of presenter metadata, and fields and limit are examples of compiler metadata. Using functions in metadata values allow fields to be used as transformer metadata.

Design and implementation details

```
mermaid
graph TD
```

```
Query[/GLQL Query/] --> Facade[Façade]
```

```
subgraph Compiler[Query Compiler]
  Parser[Parser] --> Analyzer[Analyzer]
  Analyzer --> Codegen[Code Generator]
end
```

```
subgraph Metadata[Metadata Parser]
  JSYAML[JS YAML]
  FieldsParser[Fields Parser]
end
```

```
Facade -- "query" --> Compiler
Facade -- "metadata" --> Metadata
Metadata -- "compiler metadata" --> Compiler
Compiler -- "compiled query" --> Executor[Executor]
Executor <--> DB[(API or Datastore)]
Executor -- "response data" --> Transformer[Transformer]
Metadata -- "transformer
      metadata" --> Transformer
Transformer -- "transformed data" --> Presenter[Presenter]
Metadata -- "presenter
      metadata" --> Presenter
Presenter --> Result((Result))
```

Façade

Serving as the entry-point to parsing, executing and presenting a GLQL query, the façade component consists of two parts: the model and the view.

The model part does all the hard work, while the view part is an abstraction to handle loading and error states, and present the data using the appropriate root presenter component in case of success.

Query Compiler

The query compiler is built with Haskell and is hosted [here](#).

It is built using Haskell Stack and is compiled to WASM, which is then deployed to [@gitlab/query-language](#) on NPM.

Syntax:

The query syntax consists primarily of logical expressions. These expressions follow the syntax of [fieldName] [<|>|=|!=|in] [value|function] [AND] [anotherExpression].

Currently supported field names to query by include: assignee, author, label, epic etc. Support for more fields will be added in later stages.

- Supported comparison operators: <, >, =, !=, in.
- Supported logical operators: Currently only AND operator is supported.
- Supported operands: Currently only currentUser() and today() are supported. Some fields like milestone and iteration support dynamic values like none, any, current or upcoming.

Components:

The query compiler is built from the ground up to support multiple targets of output. Its components include:

1. Parser: This includes a combinator parser that converts a string, including the query, into an abstract syntax tree. The AST is optimized by extracting global attributes like project or group.
1. Analyzer: A static analyzer that analyzes the query for semantic correctness.
1. Code Generator(s): Converts the analyzed query into executable code for the Executor, depending on the target. For GraphQL target, it generates a GraphQL query.

Examples:

1. All bugs created for my stage in the last 28 days

label = ("devops::plan", "type::bug") and created > -28d

2. Open issues assigned to me that haven't been updated in a week

status = "opened" and assignee = currentUser() AND updated < -7d

3. Issues late in the build stage of the upcoming milestone

milestone = upcoming and label in ("workflow::in review", "workflow::verification")

Metadata Parser

Metadata for a GraphQL query is written in a combination of YAML and a special syntax to allow transformations to happen on top of the query result. The metadata parser consists of the following:

1. JS YAML: Converts a YAML block into a JSON config.
1. Fields Parser:
A combinator parser to allow dynamically creating new fields in the transformer later.

Syntax: Currently supported options:

- display: How to display the data. Currently supported options: table, list or orderedList. Default: table.
- limit: How many items to display.
- fields: A comma separated value of fields. If not provided, only title field is

included by default.

The fields option may also include functions to derive new columns. For example: labels("workflow::") can be used to derive a new column with just workflow labels extracted.

This capability to generate derived columns allows us in the future to derive custom computed columns, that are a function of existing custom fields. For example: div(mult(reach, impact, confidence), effort) as "RICE Score".

Executor

The executor is a simple module that executes the query compiled by the query compiler for the target platform, which is currently GraphQL.

Finally, the executor executes the GraphQL query and returns back the response data and the parsed YAML front matter config.

Transformer

The transformer module is responsible for two things:

- Normalizing the data for the presenter.
- Transforming data as requested by the user, for example, to derive new columns.

In a future iteration, the transformer module can also be used to aggregate data.

The transformer takes response data returned by the executor and some metadata, and generates a transformed output that can be passed on to the presenter.

Presenter

The presenter takes the output of the query transformed by the transformer and presents it by selecting appropriate presenter components.

The presenter presents the root object according to the display parameter provided (table or list), and represents the values of fields returned by the executor using appropriate object presenters. Each presenter is a Vue component that takes the data it needs to present and optionally a config object containing presentation options. The presenter module makes sure all data and fields are presented using appropriate presenters.

The result of the GraphQL query is recursively mapped to a presenter.

Currently supported presenters:

Data presenters: This includes List and Table presenters. They represent a collection of, say, issues, as a list or table of data.

Both presenters include a config prop containing fields and caption telling the presenter what fields to display, and an optional caption to render alongside the presented data.

Object presenters: These are of two types:

1. Generic object presenters: This includes presenters for generic data types like

Null,
Text,
Bool,
Time,
Link and
Collection.

1. Null: Displays null values as None. Useful for fields that have no data set, like health, or due.
2. Text: For all text or numeric fields like title or weight.
3. Bool: Represents true or false values as Yes or No. Applies to fields like confidential.
4. Time: Represents all non-null time field values like created or updated as X days ago, or in X days.
5. Link: For objects that don't have presenters yet but have a title field and a webUrl or webPath fields available, it is represented as a link.
6. Collection: When a field contains a collection of other objects, for example assignees or labels, it is represented as a space separated list of presenters.

1. GitLab reference presenters: This includes presenters for various gitlab objects like

Label,
Issue,
Milestone,
State,
User and
Health.

1. Label: Presents a GitLab group or project label with its appropriate colors with a link to issues in that label in the current project.
2. Issue: Presents an issue like it is rendered in Markdown: with its title, link and issue ID, and a popover containing additional information of the issue.
3. Milestone: Similar to the issue presenter, presenting a milestone, with its title, link and a popover containing additional information.
4. State: Presents the issue status, whether open or closed, as a badge.
5. User: Presents a user's username with a link to their profile, and a popover containing more information about the user.
6. Health: Presents the issue's health status: either of on track, needs attention or at risk as a badge.

Extensibility

GLQL's architecture is designed with extensibility as a core principle, allowing for seamless integration of new features and capabilities. This modular design enables the system to evolve with GitLab's growing needs while maintaining consistency and reliability.

Key Extensibility Features

1. New Data Sources

- To support new APIs or datastores:
 - Implement new Code Generators tailored to the specific data source
 - Develop corresponding transformers to normalize the data returned by the API
- Existing Parser and Presenter components can be reused, ensuring consistency in query input and output formats

1. New Query Objects

- To add support for querying new GitLab objects:
 - Create new Analyzers to validate object-specific query semantics
 - Implement dedicated Code Generators for the new API or data source
 - Develop specialized Transformers to process the new object types
 - Extend or create new Presenters if unique display formats are required

1. Component Modularity

- Each component (Parser, Analyzer, Code Generator, Transformer, Presenter) is designed as a standalone module
- This modularity allows for:
 - Independent updates and improvements to individual components
 - Easy replacement or addition of components without affecting the entire system

1. Metadata Extensibility

- The Metadata Parser can be extended to support new metadata fields, display formats or functions.
- This allows for the introduction of new query options or presentation styles without altering the core query language

Portability

GLQL's current implementation as a frontend-only solution demonstrates its initial focus on user interaction and rapid prototyping. However, the system's architecture is designed with portability in mind, allowing for future adaptations and optimizations.

Current Implementation

- Compiler: Written in Haskell, leveraging its strong type system and pattern matching capabilities
- Other modules (e.g., Transformer): Implemented in JavaScript for frontend integration

Future Portability Options

1. Language Migration

- The system can be ported to more widely adopted languages such as Rust or Go
- Benefits:
 - Improved performance and memory management
 - Broader developer ecosystem for maintenance and contributions
 - Potential for both frontend (using WASM) and backend implementations

2. Backend Integration

- GLQL can be integrated with GitLab's Ruby on Rails backend
- Advantages:
 - Direct access to GitLab's data models and business logic
 - Reduced network overhead for complex queries
 - Enhanced security and access control management

3. Cross-Platform Compatibility

- The modular design allows for implementation across various platforms:
 - Web browsers (current implementation)
 - Command-line interfaces
 - IDE plugins

By prioritizing portability in its design, GLQL ensures long-term flexibility and adaptability, allowing it to evolve alongside GitLab's architecture and meet emerging performance and integration requirements.

SECTION: engineering/architecture/design-
documents/google_artifact_registry_integration/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab and Google Cloud have recently announced a partnership to combine the unique capabilities of their platforms.

As highlighted in the announcement, one key goal is the ability to "use Google's Artifact Registry with GitLab pipelines and packaging to create a security data plane".

The initial step toward this goal is to allow users to configure a new Google Artifact Registry (abbreviated as GAR from now on) project integration and display container image artifacts in the GitLab UI.

Motivation

Refer to the announcement blog post for more details about the motivation and long-term goals of the GitLab and Google Cloud partnership.

Regarding the scope of this design document, our primary focus is to fulfill the Product requirement of providing users with visibility over their container images in GAR. The motivation for this specific goal is rooted in foundational research on the use of external registries as a complement to the GitLab container registry (internal).

Since this marks the first step in the GAR integration, our aim is to achieve this goal in a way that establishes a foundation to facilitate reusability in the future. This groundwork could benefit potential future expansions, such as support for additional artifact formats (npm, Maven, etc.), and features beyond the Package stage (e.g., vulnerability scanning, deployments, etc.).

Goals

- Allow GitLab users to configure a new project integration for connecting to GAR.
- Limited to a single top-level GAR repository per GitLab project.
- Limited to GAR repositories in Standard mode. Support for Remote and Virtual repository modes (both in Preview) is a stretch goal.
- Limited to GAR repositories of format Container images.
- Use a Google Cloud service account provided by the GitLab project owner/maintainer to interact with GAR.
- Allow GitLab users to list container images under the connected GAR repository, including sub-repositories. The list should be paginable and sortable.
- For each listed image, display its URI, list of tags, size, digest, upload time, media type, build time, and update time, as documented here.
- Listing container images under the connected GAR repository is restricted to users with Reporter+ roles.

Non-Goals

While some of these may become goals for future iterations, they are currently out of scope:

- Create, update and delete operations.
- Connecting to multiple (top-level) GAR repositories under the same project.
- Support for repository formats beyond container images.
- Support for other Identity and Access Management (IAM) permissions/credentials beyond service accounts.
- GAR cleanup policies.
- Filtering the images list by their attributes (name or value). The current GAR API does not support filtering.
- Artifact analysis and vulnerability scanning.

Proposal

Design and Implementation Details

Project Integration

A new project integration for GAR will be created. Once enabled, this will display a new "Google Artifact Registry" item in the "Operate" section of the sidebar. This is also where the Harbor integration is displayed if enabled.

The GAR integration can be enabled by project owner/maintainer(s), who must provide four configuration parameters during setup:

- GCP project ID: The globally unique identifier for the GCP project where the target GAR

repository lives.

- Repository location: The GCP location where the target GAR repository lives.
- Repository name: The name of the target GAR repository.
- GCP service account key: The content (not the file) of the service account key in JSON format (sample).

Authentication

The integration is simplified by using a single GCP service account for the integration. Users retain the ability to audit usage of this service account on the GCP side and revoke permissions if/when necessary.

The service account key provided during the integration setup must be granted at least with the Artifact Registry Reader role in the target GCP project.

Saving the (encrypted) service account key JSON content in the backend allows us to easily grab and use it to initialize the GAR client (more about that later). Providing the content of the key file instead of uploading it is similar to what we do with users' public SSH keys.

As previously highlighted, access to the GAR integration features is restricted to users with Reporter+ roles.

Resource Mapping

For the GitLab container registry, repositories within a specific project must have a path that matches the project full path. This is essentially how we establish a resource mapping between GitLab Rails and the registry, which serves multiple purposes, including granular authorization, scoping storage usage to a given project/group/namespace, and more.

Regarding the GAR integration, since there is no equivalent entities for GitLab project/group/namespace resources on the GAR side, we aim to simplify matters by allowing users to attach any GAR repository to any GitLab project, regardless of their respective paths. Similarly, we do not plan to restrict the attachment of a particular GAR repository to a single GitLab project. Ultimately, it is up to users to determine how to organize both datasets in the way that best suits their needs.

GAR API

GAR provides three APIs: Docker API, REST API, and RPC API.

The Docker API

is based on the Docker Registry HTTP API V2, now superseded by the OCI Distribution Specification API (from now on referred to as OCI API). This API is used for pushing/pulling images to/from GAR and also provides some discoverability operations. Refer to Alternative Solutions for the reasons why we don't intend to use it.

Among the proprietary GAR APIs, the REST API

provides basic functionality for managing repositories. This includes
list
and get
operations for container image repositories, which could be used for this integration.
Both operations return the same data structure, represented by the
DockerImage
object, so both provide the same level of detail.

Last but not least, there is also an RPC API,
backed by gRPC and Protocol Buffers. This API provides the most functionality,
covering all GAR features. From the available operations, we can make use of the
ListDockerImagesRequest
and GetDockerImageRequest
operations. As with the REST API, both responses are composed of
DockerImage
objects.

Between the two proprietary API options, we chose the RPC one because it
provides support not only for the operations we need today but also offers
better coverage of all GAR features, which will be beneficial in future
iterations. Finally, we do not intend to make direct use of this API but rather
use it through the official Ruby client SDK. See Client SDK below for more details.

Backend Integration

This integration will need several changes on the backend side of the rails project. See the
backend page for additional details.

UI/UX

This integration will include a dedicated page named "Google Artifact Registry,"
listed under the "Operate" section of the sidebar. This page will enable users to
view the list of all container images in the configured GAR repository. See the
UI/UX page for additional details.

GraphQL APIs

TODO: Describe any GraphQL APIs or changes to existing APIs that will be needed for this
integration.

Alternative Solutions

Use Docker/OCI API

One alternative solution considered was to use the Docker/OCI API provided by
GAR, as it is a common standard for container registries. This approach would
have allowed GitLab to reuse existing logic
for connecting to container registries, which could potentially speed up
development. However, there were several drawbacks to this approach:

- Authentication Complexity: The API requires authentication tokens, which need to be requested at the login endpoint. These tokens have limited validity, adding complexity to the authentication process. Handling expiring tokens would have been necessary.
- Limited Focus: The API is solely focused on container registry objects, which does not align with the goal of creating a flexible integration framework for adopting additional GAR artifacts (e.g. package registry formats) down the road.
- Discoverability Limitations: The API has severe limitations when it comes to discoverability, lacking features like filtering or sorting.
- Multiple Requests: To retrieve all the required information about each image, multiple requests to different endpoints (listing tags, obtaining image manifests, and image configuration blobs) would have been necessary, leading to a 1+N performance issue.

GitLab had previously faced significant challenges with the last two limitations, prompting the development of a custom GitLab container registry API to address them. Additionally, GitLab decided to deprecate support for connecting to third-party container registries using the Docker/OCI API due to these same limitations and the increased cost of maintaining two solutions in parallel. As a result, there is an ongoing effort to replace the use of the Docker/OCI API endpoints with custom API endpoints for all container registry functionalities in GitLab.

Considering these factors, the decision was made to build the GAR integration from scratch using the proprietary GAR API. This approach provides more flexibility and control over the integration and can serve as a foundation for future expansions, such as support for other GAR artifact formats.

SECTION: engineering/architecture/design-
documents/google_artifact_registry_integration/backend.md

Client SDK

To interact with GAR we will make use of the official GAR Ruby client SDK. By default, this client will use the RPC version of the Artifact Registry API.

To build the client, we will need the service account key.

Interesting functions

For the scope of this blueprint, we will need to use the following functions from the Ruby client:

- getrepository. API documentation. This will return a single Repository.
- listdockerimages. API documentation. This will return a list of DockerImage.

- `getdockerimage`. API documentation. This will return a single `DockerImage`.

Limitations

Filtering is not available in `listdockerimages`. In other words, we can't filter the returned list (for example on a specific name). However, ordering on some columns is available.

In addition, we can't point directly to a specific page. For example, directly accessing page 3 of the list of Docker images without going first through page 1 and 2.

We can't build this feature on the GitLab side because this will require to walk through all pages and we could hit a situation where we need to go through a very large amount of pages.

Exposing the client

It would be better to centralize the access to the official Ruby client. This way, it's very easy to check for permissions.

We suggest having a custom client class located in `GoogleCloudPlatform::ArtifactRegistry::Client`. That class will need to require a `User` and a `Integrations::GoogleCloudPlatform::ArtifactRegistry` (see Project Integration).

The client will then need to expose three functions: `repository`, `dockerimages` and `dockerimage` that will be mapped to the similarly name functions of the official client.

Before calling the official client, this class will need to check the user permissions. The given `User` should have `readgcpartifactregistryrepository` on the Project related with the `Integrations::GoogleCloudPlatform::ArtifactRegistry`.

Lastly, to set up the official client, we will need to properly set:

- the timeout.
- the retry policy.

For these, we can simply either use the default values if they are ok or use fixed values.

New permission

We will need a new permission on the Project policy:

- `readgcpartifactregistryrepository` granted to at least reporter users.

Project Integration

We will need to build a new project integration with the following properties:

- `googleprojectid` - the Google project ID. A simple string.
- `googlelocation` - the Google location. A simple string.
- `repositories` - an array of repository names (see below).
- `jsonkey` - the service account JSON. A string but displayed as a text area.
- `jsonkeybase64` - the service account JSON, encoded with base64. Value set from `jsonkey`.

We will also have derived properties:

- repository- the repository name. Derived from repositories.

repositories is used as a way to store the repository name in an array. This is to help with a future follow up where multiple repositories will need to be supported. As such, we store the repository name into an array and we create a repository property that is the first entry of the array. By having a repository single property, we can use the frontend helpers as array values are not supported in project integrations.

We also need the base64 version of the jsonkey. This is required for the CI/CD variables.

Regarding the class name, we suggest using `Integrations::GoogleCloudPlatform::ArtifactRegistry`. The `Integrations::GoogleCloudPlatform` namespace allows us to have possible future other integrations for the other services of the Google Cloud Platform.

Regarding the configuration test, we need to get the repository info on the official API (method `getrepository`). The test is successful if and only if, the call is successful and the returned repository has the format `DOCKER`.

GraphQL APIs

The UI will basically have two pages: listing Docker images out of the repository configured in the project integration and show details of a given Docker image.

In order to support the other repository formats in follow ups, we choose to not map the official client function names in GraphQL fields or methods but rather have a more re-usable approach.

All GraphQL changes should be marked as alpha.

First, on the `ProjectType`, we will need a new field `googlecloudplatformartifactregistryrepositoryartifacts`. This will return a list of an abstract new type: `GoogleCloudPlatform::ArtifactRegistry::ArtifactType`. This list will have pagination support. Ordering options will be available.

We will have `GoogleCloudPlatform::ArtifactRegistry::DockerImage` as a concrete type of `GoogleCloudPlatform::ArtifactRegistry::ArtifactType` with the following fields:

- name. A string.
- uri. A string.
- imagesizebytes. A integer.
- uploadtime. A timestamp.

Then, we will need a new query `Query.googlecloudplatformregistryregistryartifactdetails` that given a name of a `GoogleCloudPlatform::ArtifactRegistry::DockerImage` will return a single `GoogleCloudPlatform::ArtifactRegistry::ArtifactDetailsType` with the following fields:

- all fields of `GoogleCloudPlatform::ArtifactRegistry::ArtifactType`.
- tags. An array of strings.

- mediatype. A string.
- buildtime. A timestamp.
- updatedtime. A timestamp.

All GraphQL changes will require users to have the `readgcpartifactregistryrepository` permission.

CI/CD variables

Similar to the Harbor integration, once users activates the GAR integration, additional CI/CD variables will be automatically available if the integration is enabled. These will be set according to the requirements described in the documentation:

- `GCPARTIFACTREGISTRYURL`: This will be set to `https://LOCATION-docker.pkg.dev`, where `LOCATION` is the GCP project location configured for the integration.
- `GCPARTIFACTREGISTRYPROJECTURI`: This will be set to `LOCATION-docker.pkg.dev/PROJECT-ID`. `PROJECT-ID` is the GCP project ID of the GAR repository configured for the integration.
- `GCPARTIFACTREGISTRYPASSWORD`: This will be set to the base64-encode version of the service account JSON key file configured for the integration.
- `GCPARTIFACTREGISTRYUSER`: This will be set to `jsonkeybase64`.

These can then be used to log in using docker login:

shell

```
docker login -u $GCPARTIFACTREGISTRYUSER -p $GCPARTIFACTREGISTRYPASSWORD
$GCPARTIFACTREGISTRYURL
```

Similarly, these can be used to download images from the repository with docker pull:

shell

```
docker pull $GCPARTIFACTREGISTRYPROJECTURI/REPOSITORY/myapp:latest
```

Finally, provided that the configured service account has the Artifact Registry Writer role, one can also push images to GAR:

shell

```
docker build -t $GCPARTIFACTREGISTRYREPOSITORYURI/myapp:latest .
docker push $GCPARTIFACTREGISTRYREPOSITORYURI/myapp:latest
```

For forward compatibility reasons, the repository name (`REPOSITORY` in the command above) must be appended to `GCPARTIFACTREGISTRYPROJECTURI` by the user. In the first iteration we will only support a single GAR repository, and therefore we could technically provide a variable like `GCPARTIFACTREGISTRYREPOSITORYURI` with the repository name already included. However, once we add support for multiple repositories, there is no way we can tell what repository a user will want to target for a specific instruction.

SECTION: engineering/architecture/design-
documents/google_artifact_registry_integration/ui_ux.md

Structure and Organization

Unlike the GitLab container registry (and therefore the Docker Registry and OCI Distribution), GAR does not treat tags as the primary "artifacts" in a repository. Instead, the primary "artifacts" are the image manifests. For each manifest object (represented by DockerImage), there is a list of assigned tags (if any). Consequently, when listing the contents of a repository through the GAR API, the response comprises a collection of manifest objects (along with their associated tags as properties), rather than a collection of tag objects. Additionally, due to this design choice, untagged manifests are also present in the response.

To maximize flexibility, extensibility, and maintain familiarity for GAR users, we plan to fully embrace the GAR API data structures while surfacing data in the GitLab UI. We won't attempt to emulate a "list of tags" response to match the UI/UX that we already have for the GitLab container registry.

Considering the above, there will be a view that provides a pageable and sortable list of all images in the configured GAR repository. Additionally, there will be a detail view to display more information about a single image. You can find a list of available image attributes documented here.

Designs

SECTION: engineering/architecture/design-
documents/google_cloud_platform_integration/_index.md

{{< engineering/design-document-header >}}

GitLab and Google Cloud Platform (GCP) provide complementary tooling which we are integrating via our partnership.

This design doc is not public at that time. The whole content is available in GitLab-internal project.

Who

<!-- vale gitlab.Spelling = NO -->

Who	Role
-----	-----
Sam Goldstein	Direct